

Inspired by recognized software documentation practices from arc42, C4 Model, Google, AWS, and ETH Zurich.

Model STS

[INSTITUTION OR ORGANIZATION NAME]

SOFTWARE TECHNICAL SPECIFICATION

[SYSTEM OR FEATURE NAME]

[AUTHOR OR TEAM NAME]

Technical document prepared to describe the context, requirements, architecture, contracts, validation, and operation of the [name] system. Technical owner: [name]. Product owner: [name]. Version: [x.y].

[CITY]

[YEAR]

USAGE GUIDE AND CONTENTS

COMPLETION INSTRUCTION: This is a technical report adapted to ABNT presentation standards. Use Arial 12, A4 paper, 3 cm left and top margins, and 2 cm right and bottom margins. Keep 1.5 line spacing. Tables, captions, and references may use a smaller, consistent font with single spacing. Confirm your institution's handbook before submission.

READ BEFORE COMPLETING

1. Delete bracketed text after completing it. 2. Preserve requirement and decision identifiers. 3. Include links to repositories, diagrams, contracts, tests, and runbooks. 4. Do not record passwords, tokens, or sensitive personal data. 5. Remove non-applicable sections and record the reason.

CONTENTS

Section	Content	Page
1 INTRODUCTION AND SCOPE	Problem, objective, audience, boundaries, and owners.	1
2 REQUIREMENTS AND TRACEABILITY	What must be delivered and how to validate it.	2
3 ARCHITECTURE AND SCENARIOS	C4 view, components, and solution decisions.	3
4 DATA, CONTRACTS AND INTEGRATIONS	Data, APIs, events, and external dependencies.	4
5 QUALITY, SECURITY AND TESTING	Goals, risks, controls, and evidence.	5
6 DEPLOYMENT AND OPERATIONS	Deployment, rollback, observability, and support.	6
7 DECISIONS AND REFERENCES	ADRs, checklist, and consulted sources.	7

COMPLETION INSTRUCTION: Content basis: arc42 for architecture, C4 for diagrams, Google for writing, AWS ADRs for decisions, and ETH Zurich for separating requirements, design, and testing. The presentation structure was adapted from a technical report, not a monograph.

1 INTRODUCTION AND SCOPE

COMPLETION INSTRUCTION: Explain the problem and intended outcome first. The reader should understand why the work matters before reading about technology.

1.1 Identification and objective

System or feature	[Name and one sentence describing the deliverable.]
Current problem	[What pain, risk, or opportunity exists? Example: manual order confirmation takes up to two days.]
Objective	[Measurable outcome. Example: confirm orders within five minutes in 95% of cases.]
Audience	[Users, teams, and affected systems.]
Owners	[Author, technical reviewer, product owner, and operations owner.]

1.2 Scope, exclusions, and assumptions

In scope	Out of scope	Assumptions / dependencies
[Capabilities delivered in this version.]	[Related items not delivered.]	[APIs, teams, deadlines, data, licenses, or external standards.]

1.3 Version control

Version	Date	Author	Change	Approval
0.1	[AAAA-MM-DD]	[Name]	[First draft]	[Pending]

Write short sentences with action verbs and observable measures. Avoid vague terms such as fast, secure, or simple without a verification criterion.

2 REQUIREMENTS AND TRACEABILITY

COMPLETION INSTRUCTION: ETH Zurich inspires this separation: requirements define what the system must do; design defines how; tests prove whether the requirement was met.

2.1 Functional requirements

ID	Requirement	Priority	Acceptance criteria
RF-01	[The system must allow the user to ...]	[High]	[Given X, when Y, then Z.]
RF-02	[...]	[Medium]	[...]

2.2 Non-functional requirements

ID	Attribute	Objective target	Evidence
RNF-01	Performance	[p95 below 300 ms for endpoint X.]	[Load test / dashboard.]
RNF-02	Availability	[99.9% monthly for service Y.]	[SLO / monitoring.]
RNF-03	Security	[Only role Z can read data W.]	[Authorization test.]
RNF-04	Accessibility	[Primary flow meets the defined criterion.]	[Audit / manual test.]

2.3 Traceability matrix

Requirement	Design / ADR	Test	Evidence
[RF-01]	[Link to section, diagram, or ADR]	[Test ID/link]	[Result/link]

Practical rule: if no test or evidence is possible, rewrite the requirement until it is clear and verifiable.

3 ARCHITECTURE AND SCENARIOS

COMPLETION INSTRUCTION: Use C4 to show the system in layers: context, containers, and components. Every diagram should include a title, date, version, and legend. Store the diagram source in the repository.

3.1 C4 context

Actors	[People or systems that initiate actions.]
Boundary	[What belongs to the system and what is external.]
Integrations	[External system - data exchanged - protocol - owner.]
Diagram	[Insert or reference Diagram 1 - System context.]

3.2 Containers and components

Element	Responsibility	Technology	Inputs / outputs
[Web / API / worker]	[One primary responsibility.]	[Language and framework.]	[HTTP, event, queue, file.]
[Database / storage]	[Data and retention.]	[Product and version.]	[Read, write, backup.]

3.3 Essential scenarios

Scenario	Steps	Expected failure and response
[Success flow]	[1. ... 2. ... 3. ...]	[Outcome, log, metric, and owner.]
[External failure]	[Timeout, retry, queue, fallback.]	[Impact, alert, and communication.]

3.4 Strategy and alternatives

Approach	[Example: modular monolith with an asynchronous job.]
Alternatives	[Alternative A - reason for rejection. Alternative B - reason.]
Constraint	[Cost, legacy, latency, team, compliance, or SLA.]

4 DATA, CONTRACTS AND INTEGRATIONS

COMPLETION INSTRUCTION: Document the system boundaries. Link to OpenAPI, AsyncAPI, JSON Schema, or another versioned contract instead of duplicating lengthy specifications.

4.1 Relevant data

Data / entity	Classification	Source and retention	Rule
[Customer, order, event.]	[Public, internal, personal, sensitive.]	[Source / period / disposal.]	[Validation, access, anonymization.]

4.2 API or event

Identifier	[POST /v1/resource] or [event.resource.created.v1].
Consumers	[Consumer applications, teams, or systems.]
Contract	[Link to OpenAPI, AsyncAPI, schema, or short example.]
Error handling and idempotency	[Codes, format, idempotency key, retry, ordering.]
Compatibility	[How changes are versioned, deprecated, and communicated.]

4.3 External integrations

System	Protocol	Authentication	Limit	Failure plan
[Name]	[REST, queue, SFTP.]	[OAuth, mTLS, secret.]	[Timeout/rate limit.]	[Retry, cache, queue, fallback.]

Never copy secrets into this document. Describe where they are stored, how they are rotated, and who may access them.

5 QUALITY, SECURITY AND TESTING

COMPLETION INSTRUCTION: Turn each concern into a goal, control, and evidence. Saying that the system is secure is not evidence; a test, configuration, or audit is.

5.1 Controls

Area	Control adopted	Evidence / owner
Access	[Authentication, authorization, least privilege.]	[Test, política e dono.]
Data	[Encryption, masking, retention, deletion.]	[Configuration and audit.]
Input	[Validation, rate limit, timeout.]	[Test and gateway.]
Availability	[SLO, backup, recovery, degradation.]	[Dashboard and runbook.]

5.2 Risks

Risk	Prob. / impact	Mitigation	Trigger / owner
[Dependency exceeds the API limit.]	[Medium / high]	[Cache, queue, contract.]	[Alert X / name.]
[...]	[...]	[...]	[...]

5.3 Test plan

Coverage	[Unit] [integration] [contract] [end-to-end] [load] [security]. Keep only the applicable tests.
Environment	[Environment, anonymized data, test accounts, and simulated dependencies.]
Release	[Required tests, review, migration, rollback plan, and acceptance.]

6 DEPLOYMENT AND OPERATIONS

COMPLETION INSTRUCTION: This section turns the design into operable software. Anyone who did not take part in the project should be able to deploy, observe, and recover the system using the recorded steps and links.

6.1 Deployment

Environments	[Local - test - staging - production. State the relevant differences.]
Pipeline	[Build - tests - scan - migration - deploy - validation. Link to CI/CD.]
Rollback	[When to trigger it, steps, data impact, and owner.]
Configuration	[Where configuration and secrets are kept; how to rotate them.]

6.2 Observability and support

Signal	What it indicates	Alert / first action	Link
[Metric, log, or trace.]	[Success or failure.]	[Threshold, severity, and first step.]	[Dashboard / runbook.]

6.3 Maintenance and evolution

Item	Reason	Priority	Next step
[Technical debt or improvement.]	[Risk or benefit.]	[High, medium, low.]	[Ticket, owner, date.]

Update this section whenever the pipeline, dashboard, SLA, dependency, or runbook changes. Linking to the source of truth reduces staleness.

7 DECISIONS AND REFERENCES

COMPLETION INSTRUCTION: Create an ADR for decisions with architectural impact. An accepted decision is not edited to rewrite history: a new ADR supersedes it.

7.1 Architecture Decision Record - ADR

Identifier	ADR-[NNN] - [title] - [Proposed / Accepted / Superseded / Rejected].
Context	[Facts, constraints, competing forces, and need.]
Decision	[We chose X to meet Y.]
Alternatives	[A and B] - [objective reasons not to choose them.]
Consequences	[Benefits, costs, risks, future work, and success measure.]
Links	[Issue, PoC, diagram, PR, and vendor documentation.]

7.2 Approval checklist

- The problem, objective, scope, and requirements are understandable without reading the code.
- The architecture shows boundaries, dependencies, and failure flows.
- Data, contracts, security, testing, and operations have a verifiable criterion or link to the source of truth.
- Relevant decisions present alternatives and consequences.
- Links work; no secrets are included; owners are defined.

REFERENCES

BRAZILIAN ASSOCIATION OF TECHNICAL STANDARDS. ABNT NBR 10719:2015: information and documentation - technical and/or scientific report - presentation. Rio de Janeiro: ABNT, 2015.

ARC42. arc42 - template for software architecture documentation. Available at: <<https://arc42.org/overview/>>. Accessed on: September 14, 2026.

BROWN, Simon. The C4 model for visualising software architecture. Available at: <<https://c4model.com/>>. Accessed on: September 14, 2026.

GOOGLE. Google developer documentation style guide. Available at: <<https://developers.google.com/style>>. Accessed on: September 14, 2026.

AMAZON WEB SERVICES. Using architectural decision records. Available at: <<https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/>>. Accessed on: September 14, 2026.

ETH ZÜRICH. Software architecture course materials. Available at: <https://se.inf.ethz.ch/courses/2011a_spring/soft_arch/>. Accessed on: September 14, 2026.